

Plan Maestro de Desarrollo

Proyecto Hill Climb AI

Versión 2.0 — Revisada y comentada

Índice

1. Objetivo General	3
2. Tecnologías del Proyecto	3
3. Arquitectura Final	4
4. Responsabilidad de Cada Archivo	4
4.1. main.py	4
4.2. config/settings.py	5
4.3. game/environment.py	5
4.4. game/terrain.py	6
4.5. game/vehicle.py	6
4.6. game/player.py	6
4.7. game/coin.py	6
4.8. game/checkpoint.py	7
4.9. game/physics.py	7
4.10. game/ui.py	7
4.11. game/camera.py	7
4.12. ai/neural_network.py	7
4.13. ai/genome.py	8
4.14. ai/genetic_algorithm.py	9
4.15. ai/trainer.py	9
4.16. ai/reward_system.py	9
4.17. utils/helpers.py	9
4.18. utils/logger.py	10
4.19. utils/math_utils.py	10
5. Persistencia	10
6. Plan de Desarrollo	10
6.1. FASE 1: Preparación	10
6.2. FASE 2: Juego Manual	11
6.3. FASE 3: Sistema de Recompensas	11
6.4. FASE 4: Estados de IA	12
6.5. FASE 5: Red Neuronal	12
6.6. FASE 6: Algoritmo Genético	12
6.7. FASE 7: Persistencia	12
6.8. FASE 8: Optimización y Extensiones (Opcional)	12

7. Reglas de Vibe Coding	13
8. Resultado Esperado	13

1. Objetivo General

Desarrollar un entorno inspirado en *Hill Climb Racing* utilizando únicamente Python. El proyecto tiene dos componentes principales:

1. **Simulación jugable** con física basada en restricciones rígidas.
2. **Entrenamiento de una IA** mediante neuroevolución para aprender a conducir automáticamente.

La simulación se ejecuta visualmente en tiempo real y muestra el aprendizaje de la IA mientras evoluciona generación tras generación.

El objetivo final NO es construir un videojuego comercial, sino un entorno de aprendizaje por refuerzo (vía neuroevolución) modular, extensible y presentable como proyecto de portafolio en ciencia de datos.

Nota de aprendizaje

La neuroevolución es una alternativa al aprendizaje por refuerzo clásico (Q-learning, PPO). En lugar de calcular gradientes para mejorar la política, se mantiene una *población* de redes neuronales, se evalúa cuál juega mejor, y los mejores se “reproducen” mutando sus pesos. Es ideal para problemas donde la recompensa es escasa o ruidosa, y se puede paralelizar bien.

2. Tecnologías del Proyecto

Instalación inicial:

```
1 pip install pygame pymunk numpy torch matplotlib
```

Función de cada librería:

- **pygame**: renderizado de gráficos 2D, captura de eventos de teclado/ratón, UI.
- **pymunk**: motor de física 2D (basado en Chipmunk). Maneja cuerpos rígidos, articulaciones, colisiones y gravedad.
- **numpy**: operaciones vectoriales rápidas (lookahead de terreno, normalización de estados).
- **torch**: implementación de la red neuronal. Aunque no usaremos backpropagation, PyTorch facilita el manejo de tensores y será útil si extiendes el proyecto a DRL más adelante.
- **matplotlib**: gráficas de evolución del fitness, distribuciones de mutación, métricas de entrenamiento.

Nota de aprendizaje

Sobre PyTorch en neuroevolución: como NO usamos gradientes, todos los tensores de la red deben crearse con `requires_grad=False`, y los forward passes deben envolverse en `with torch.no_grad():`. Esto evita que PyTorch construya el grafo computacional y mejora el rendimiento. Es una excelente oportunidad para entender cómo PyTorch separa el cálculo del autograd.

3. Arquitectura Final

La estructura NO debe modificarse durante el proyecto.

```
1 hill_climb_ai/  
2   main.py  
3   config/  
4       settings.py  
5   game/  
6       environment.py  
7       terrain.py  
8       vehicle.py  
9       player.py  
10      coin.py  
11      checkpoint.py  
12      physics.py  
13      ui.py  
14      camera.py  
15  ai/  
16      neural_network.py  
17      genome.py  
18      genetic_algorithm.py  
19      trainer.py  
20      reward_system.py  
21  utils/  
22      helpers.py  
23      logger.py  
24      math_utils.py  
25  assets/  
26      images/  
27      fonts/  
28  data/  
29      saved_models/  
30      statistics/  
31  experiments/  
32      physics_tests.ipynb  
33      reward_tests.ipynb  
34      plots.ipynb
```

4. Responsabilidad de Cada Archivo

4.1. main.py

Responsabilidades:

- Inicializar `pygame` y crear ventana.
- Parsear argumentos de línea de comandos (`-mode play|train|watch`, `-render true|false`).
- Ejecutar el game loop principal.
- Controlar FPS según el modo (60 en juego/observación, sin límite en entrenamiento headless).
- Lanzar el entrenamiento llamando a `ai/trainer.py`.

Prohibido: crear lógica física, calcular recompensas, programar IA.

4.2. config/settings.py

Contiene únicamente constantes globales. Ningún import de otros módulos del proyecto.

```
1  # === Ventana y rendering ===
2  SCREEN_WIDTH = 1280
3  SCREEN_HEIGHT = 720
4  FPS = 60
5
6  # === Fisica ===
7  GRAVITY = 900          # px/s^2
8  WHEEL_FRICTION = 1.5
9  CHASSIS_MASS = 5.0
10 WHEEL_MASS = 1.0
11
12 # === Episodio ===
13 MAX_TIME = 20.0        # segundos sin checkpoint -> muerte
14 CHECKPOINT_TIME = 10.0 # segundos que anade un checkpoint
15 TARGET_DISTANCE = 100  # px minimos para considerar avance
16
17 # === Red neuronal ===
18 NN_INPUTS = 14
19 NN_HIDDEN = [16, 12]
20 NN_OUTPUTS = 2
21 LOOKAHEAD_DISTANCES = [30, 80, 150, 250] # px frente al vehiculo
22
23 # === Algoritmo genetico ===
24 POPULATION_SIZE = 50
25 ELITISM_COUNT = 3       # mejores individuos que pasan sin mutar
26 MUTATION_RATE = 0.1     # probabilidad por peso
27 MUTATION_SIGMA = 0.2    # desviacion estandar del ruido gaussiano
28 TOURNAMENT_SIZE = 3
29 CROSSEVER_RATE = 0.7
30
31 # === Persistencia ===
32 SAVE_EVERY_N_GEN = 5
33 MODEL_DIR = "data/saved_models"
34 STATS_DIR = "data/statistics"
```

Nota de aprendizaje

Todas las magnitudes con unidad están comentadas. Esto es importante en física simulada: confundir píxeles/segundo con metros/segundo es la causa #1 de comportamientos raros en pymunk.

4.3. game/environment.py

Archivo central del juego. Orquesta todos los componentes.

Responsabilidades:

- Inicializar terreno, vehículo, jugador, monedas, checkpoints.
- Llamar a `physics.py` para avanzar la simulación un `dt`.
- Verificar condición de muerte (jugador toca el suelo o tiempo expirado).
- Actualizar score y gestionar colecciones (monedas, checkpoints alcanzados).
- Exponer `get_state()` que devuelve el vector de 14 entradas para la IA.
- Exponer `step(action)` (interfaz tipo Gym): recibe acción, avanza un frame, retorna (`state`, `reward`, `done`, `info`).

4.4. game/terrain.py

Generación del mapa.

Debe:

- Crear colinas y pendientes manteniendo continuidad.
- Aceptar una **semilla** (seed) para reproducibilidad.
- Exponer `height_at(x)` para consultar altura del terreno en cualquier coordenada x (necesario para el lookahead de la red).
- Exponer `slope_at(x)` para consultar pendiente local.

Primera versión: terreno fijo (lista de puntos predefinida).

Versión posterior: terreno procedural por funciones sinusoidales superpuestas (Perlin noise opcional).

Nota de aprendizaje

La semilla del terreno debe ser **idéntica para todos los individuos de una misma generación**. Si cada uno enfrenta un terreno distinto, el fitness no es comparable. Se cambia la semilla solo al pasar a la siguiente generación (o cada N generaciones).

4.5. game/vehicle.py

Implementación física del vehículo en pymunk.

Debe contener:

- Chasis (cuerpo rígido rectangular).
- Dos ruedas (cuerpos circulares unidos al chasis por articulaciones tipo `pin joint` o `groove joint`).
- Motor en cada rueda con torque controlable.
- Tracking de inclinación y velocidad angular del chasis.

Interfaz mínima:

```
1 def accelerate(intensity: float) -> None: ...      # intensity in [0, 1]
2 def brake(intensity: float) -> None: ...           # intensity in [0, 1]
3 def update(dt: float) -> None: ...
4 def get_telemetry() -> dict: ...                   # vx, vy, angle, omega, contacts
```

4.6. game/player.py

Representa al conductor encima del chasis.

Regla principal: si el cuerpo del conductor toca el suelo, el episodio termina (`done = True`).

Implementado como un cuerpo rígido pequeño unido al chasis por un `pivot joint` rígido. La colisión con el terreno se detecta usando `collision_handlers` de pymunk.

4.7. game/coin.py

Cada moneda contiene:

- `position`: (x, y) en coordenadas del mundo.
- `value`: puntos otorgados al recogerla (por defecto 1).
- `collected`: booleano.

Las monedas se distribuyen por el terreno con espaciado pseudo-aleatorio.

4.8. game/checkpoint.py

Cada checkpoint:

- Agrega CHECKPOINT_TIME segundos al contador del episodio.
- Otorga una recompensa fija (+200).
- Reinicia el contador de inactividad.

4.9. game/physics.py

Antes ambiguo, ahora con rol definido.

Responsabilidades:

- Crear y mantener el `pymunk.Space` global.
- Configurar gravedad, damping y constantes de simulación desde `settings.py`.
- Exponer `step(dt)` que avanza la simulación.
- Registrar `collision_handlers` (vehículo-suelo, jugador-suelo, vehículo-moneda).

Prohibido: dibujar nada, contener lógica de juego.

4.10. game/ui.py

Renderiza información sobre la pantalla.

Mostrar:

- Score actual, generación, distancia, tiempo restante, mejor fitness histórico.
- Mini-gráfica de evolución del fitness (opcional, post fase 6).

También: botones de acelerar/frenar (modo manual).

4.11. game/camera.py

Sigue al vehículo horizontalmente con un pequeño *lerp* para suavizar.

NO debe contener lógica de juego. Solo transformaciones de coordenadas mundo → pantalla.

4.12. ai/neural_network.py

Red neuronal feedforward implementada en PyTorch.

Arquitectura:

14 entradas → 16 (tanh) → 12 (tanh) → 2 (sigmoide)

Entradas (vector de 14 dimensiones):

1. Velocidad horizontal v_x normalizada.
2. Velocidad vertical v_y normalizada.
3. Ángulo del chasis θ (radianes, en $[-\pi, \pi]$).
4. Velocidad angular ω .
5. Contacto rueda izquierda (0 o 1).

6. Contacto rueda derecha (0 o 1).
7. Altura terreno relativa a +30 px frente al vehículo.
8. Altura terreno relativa a +80 px.
9. Altura terreno relativa a +150 px.
10. Altura terreno relativa a +250 px.
11. Pendiente del terreno en la posición actual.
12. Δx a moneda más cercana (normalizado).
13. Δy a moneda más cercana (normalizado).
14. Tiempo restante normalizado (0 a 1).

Salidas:

1. Intensidad de aceleración (sigmoide, [0,1]).
2. Intensidad de frenado (sigmoide, [0,1]).

Importante: la red debe crearse con `requires_grad=False` en todos sus parámetros, y los forward passes envolverse en `torch.no_grad()`.

```

1 import torch
2 import torch.nn as nn
3
4 class PolicyNet(nn.Module):
5     def __init__(self, n_in=14, hidden=[16, 12], n_out=2):
6         super().__init__()
7         layers = []
8         prev = n_in
9         for h in hidden:
10             layers += [nn.Linear(prev, h), nn.Tanh()]
11             prev = h
12         layers += [nn.Linear(prev, n_out), nn.Sigmoid()]
13         self.net = nn.Sequential(*layers)
14         # Desactivar gradientes (no usamos backprop)
15         for p in self.parameters():
16             p.requires_grad = False
17
18     def forward(self, x):
19         with torch.no_grad():
20             return self.net(x)

```

4.13. ai/genome.py

Un **Genome** es un individuo de la población. Contiene:

- Su red neuronal (PolicyNet).
- Su fitness (inicialmente 0).
- Métodos: `get_weights()`, `set_weights(vector)`, `mutate()`, `copy()`.

Los pesos se manejan como un **vector plano (1D)** de NumPy para facilitar crossover y mutación. La conversión vector $\leftrightarrow$ red se hace con `torch.nn.utils.parameters_to_vector` y `vector_to_parameters`.

4.14. ai/genetic_algorithm.py

Implementa los operadores evolutivos.

Selección: torneo de tamaño `TOURNAMENT_SIZE`.

Crossover: uniforme, gen por gen (probabilidad 50% de tomar el peso del padre A o B).

Mutación: para cada peso, con probabilidad `MUTATION_RATE`, sumar ruido gaussiano $\mathcal{N}(0, \sigma)$ con $\sigma = \text{MUTATION_SIGMA}$.

Elitismo: los `ELITISM_COUNT` mejores individuos pasan sin modificación a la siguiente generación. Esto evita perder al mejor genoma por mala suerte en crossover.

Nueva población: elite + descendencia hasta completar `POPULATION_SIZE`.

4.15. ai/trainer.py

Orquestador del entrenamiento.

```
1 Crear poblacion inicial (o cargar desde disco)
2 loop generaciones:
3     para cada genoma:
4         ejecutar episodio en environment
5         calcular fitness final
6     ordenar poblacion por fitness
7     guardar metricas
8     si generacion % SAVE_EVERY_N_GEN == 0:
9         guardar poblacion
10    aplicar operadores geneticos para crear siguiente generacion
```

4.16. ai/reward_system.py

Define la función de fitness del episodio.

Fitness final del episodio:

$$\text{fitness} = d_{\text{máx}} + 50 \cdot n_{\text{monedas}}$$

donde $d_{\text{máx}}$ es la distancia horizontal máxima alcanzada y n_{monedas} es la cantidad de monedas recogidas.

Recompensa instantánea (para debugging y análisis, no para el GA):

```
1 reward_step = 0
2 reward_step += delta_distance * 0.1
3 reward_step += 50 if coin_collected else 0
4 reward_step += 200 if checkpoint_reached else 0
5 reward_step -= 200 if player_touched_ground else 0
```

Nota de aprendizaje

Distinción clave: las recompensas instantáneas (`reward_step`) son útiles si después extiendes el proyecto a Q-learning o PPO. Para el algoritmo genético solo importa el **fitness final del episodio**, que es un único escalar por individuo.

4.17. utils/helpers.py

Funciones de propósito general: conversión de coordenadas, normalización de vectores, clamp, lerp.

4.18. utils/logger.py

Logger del entrenamiento.

Responsabilidades:

- Escribir un `.csv` por generación con: generación, fitness máx, fitness medio, fitness mín, desviación estándar, mejor distancia, mejor cantidad de monedas.
- Imprimir en consola un resumen formateado por generación.

4.19. utils/math_utils.py

Funciones matemáticas reusables: cálculo de pendiente discreta, distancia euclidiana, normalización min-max, clipping de ángulos a $[-\pi, \pi]$.

5. Persistencia

El entrenamiento NO puede perderse. El sistema debe ser robusto a cierres inesperados.

Qué guardar:

- Vector de pesos del mejor individuo (`best_genome.pt`).
- Población completa en formato pickle (`population_gen_N.pkl`).
- Métricas históricas (`stats.csv`).
- Número de generación actual.

Estructura de carpetas:

```
1 data/  
2     saved_models/  
3         best_genome.pt  
4         population_gen_5.pkl  
5         population_gen_10.pkl  
6         ...  
7     statistics/  
8         stats.csv
```

Política de guardado:

```
1 if generation % SAVE_EVERY_N_GEN == 0:  
2     save_population(generation)  
3     save_best_genome()  
4 save_stats_row(generation, metrics) # cada generacion
```

Al iniciar:

```
1 if exists(latest_population_file):  
2     population, start_gen = load_population()  
3 else:  
4     population = create_random_population()  
5     start_gen = 0
```

6. Plan de Desarrollo

6.1. FASE 1: Preparación

Objetivo: crear la estructura completa del proyecto.

Humano:

1. Crear repositorio y carpeta raíz.
2. Crear todas las subcarpetas y archivos vacíos según la arquitectura.
3. Crear entorno virtual e instalar librerías.
4. Inicializar git y hacer primer commit.

IA: aún no programa. Solo verifica que la arquitectura esté correctamente creada.

Resultado esperado: proyecto vacío pero perfectamente organizado.

6.2. FASE 2: Juego Manual

Objetivo: construir la simulación sin IA, jugable con teclado.

Orden estricto de implementación:

1. `config/settings.py` (constantes).
2. `game/physics.py` (Space de pymunk).
3. `game/terrain.py` (terreno fijo primero).
4. `game/vehicle.py` (chasis + ruedas + torque).
5. `game/player.py` (conductor + colisión con suelo).
6. `game/camera.py` (seguimiento horizontal).
7. `game/ui.py` (HUD básico).
8. `game/environment.py` (integración de todo).
9. `main.py` (game loop).

Regla obligatoria: probar cada módulo antes de continuar al siguiente.

Resultado esperado: se puede jugar manualmente con teclado (flechas izq/der o A/D), el vehículo se mueve sobre el terreno, la cámara sigue, y si el conductor toca el suelo el episodio termina.

6.3. FASE 3: Sistema de Recompensas

Añadir:

- `game/coin.py` con distribución de monedas.
- `game/checkpoint.py` con extensión de tiempo.
- Score y tiempo en `ui.py`.
- Esqueleto de `ai/reward_system.py` (aún sin IA).

Humano: verificar manualmente que recoger monedas suma score, que pasar por checkpoints extiende tiempo, y que el episodio termina correctamente al agotar tiempo o tocar suelo.

Resultado esperado: juego completo y funcional sin IA.

6.4. FASE 4: Estados de IA

Implementar en `environment.py` la función `get_state()` que retorne el vector de 14 entradas.

Humano: ejecutar el juego manualmente y graficar (en `experiments/plots.ipynb`) cómo varían las 14 variables a lo largo de un episodio. Verificar:

- Que los rangos sean razonables (idealmente $[-1, 1]$ tras normalizar).
- Que las variables de lookahead respondan correctamente al terreno.
- Que los contactos de ruedas sean estables (no parpadeen).

6.5. FASE 5: Red Neuronal

Implementar `ai/neural_network.py` y `ai/genome.py`.

Prueba clave: crear una red con pesos aleatorios, conectarla al juego, y verificar que el vehículo se mueva (caóticamente, pero se mueva).

Resultado esperado: la IA controla el vehículo aunque sin aprender aún.

6.6. FASE 6: Algoritmo Genético

Implementar `ai/genetic_algorithm.py` y `ai/trainer.py`.

Pruebas obligatorias:

- Ejecutar 5 generaciones cortas (población pequeña) y verificar que el fitness promedio aumente.
- Graficar curva de evolución (mejor, promedio, peor por generación).
- Verificar elitismo: el mejor individuo nunca empeora entre generaciones.

Resultado esperado: aprendizaje visible. Tras ~30–50 generaciones la IA debe avanzar consistentemente.

6.7. FASE 7: Persistencia

Agregar guardado y carga en `trainer.py`.

Prueba de robustez:

1. Entrenar 20 generaciones.
2. Cerrar el programa.
3. Reiniciar: el entrenamiento debe continuar desde la generación 20.
4. Cargar el mejor genoma en modo `watch` y observar su desempeño.

6.8. FASE 8: Optimización y Extensiones (Opcional)

Funcionalidades opcionales para enriquecer el proyecto:

- **Modo headless de entrenamiento.** Flag `-render=False` que desactive `pygame.display.update()`. Permite correr cientos de generaciones rápido y solo renderizar cuando se quiere observar.
- **Evaluación en paralelo.** Múltiples vehículos en el mismo `pymunk.Space` con filtros de colisión (no chocan entre sí). Acelera enormemente la evaluación de la población.
- **Terreno procedural.** Suma de sinusoides o Perlin noise con semilla controlada.

- **NEAT (NeuroEvolution of Augmenting Topologies).** Evoluciona la topología además de los pesos. Más potente pero más complejo. Existe la librería `neat-python`, o se puede implementar desde cero como ejercicio.
- **Obstáculos dinámicos** (rocas, rampas, gaps).
- **Métricas avanzadas:** diversidad genética (varianza de pesos en la población), tasa de convergencia, análisis de neuronas más activas.
- **Guardado automático del mejor video** (grabar el episodio del mejor genoma de cada generación con `imageio`).

7. Reglas de Vibe Coding

Estas reglas son obligatorias y aplican a cada interacción con Claude Code.

Nunca pedir:

- “Hazme todo el proyecto.”
- “Implementa varias fases a la vez.”

Siempre pedir:

- “Explicame cómo funciona X antes de implementarlo.”
- “Implementa solamente `terrain.py` usando la interfaz ya existente.”
- “Muéstrame un ejemplo mínimo de Y antes de integrarlo.”

Reglas obligatorias:

1. **Un archivo por iteración.** Nunca dos a la vez.
2. **Integrar y probar antes de seguir.** Cada módulo nuevo se ejecuta y se verifica visualmente o con `prints/asserts`.
3. **Commit por módulo.** Cada archivo terminado y probado es un commit.
4. **Preguntar el “por qué” antes del “cómo”.** Antes de copiar código generado, pedir que se explique la decisión de diseño.
5. **Comentarios pedagógicos.** Pedir a la IA que comente el código a nivel de aprendizaje, no solo a nivel de documentación.

8. Resultado Esperado

Al finalizar el proyecto se obtiene:

- Un juego tipo Hill Climb funcional, jugable manualmente.
- Una IA entrenada por neuroevolución capaz de avanzar consistentemente y recoger monedas.
- Visualización en tiempo real del aprendizaje generación tras generación.
- Guardado persistente del entrenamiento, robusto a reinicios.
- Arquitectura modular y limpia, fácil de extender.
- Notebooks de experimentos con gráficas de evolución del fitness.

- Un proyecto sólido de portafolio en ciencia de datos / RL aplicado.

Documento revisado. Versión 2.0. Listo para la fase de implementación.